

FAIR WINDS QUANTITATIVE

Investing Analyst v3

A risk-disciplined, backtest-first algorithmic trading platform.

Technical Architecture & Investment Thesis White Paper.

SOURCE FILES

98

Python + TypeScript

TEST SUITE

183

all passing

EQUITY UNIVERSE

~4,400

scanned each cycle

Hans Seraphim · Fair Winds Quantitative

hans@legnoai.com · fairwinds.live

Live dashboard: dashboard.fairwinds.live

Source: github.com/hansseraphim-s/investing-analyst-v3

May 2026

Contents

1.	Executive summary	3
2.	Investment thesis	3
3.	System architecture	4
4.	Trading strategy stack	5
5.	Risk management layers	6
6.	A trading day, step by step	7
7.	Validation methodology	8
8.	Technology stack & infrastructure	9
9.	Honest limitations & caveats	9

1. Executive summary

Investing Analyst v3 is a production-grade algorithmic trading platform built for sophisticated retail and family-office investors. The system pairs a Python trading agent with a real-time, colleague-shareable dashboard. It is **backtest-first, risk-disciplined, and explicitly transparent**: every parameter is documented, every promotion from paper to live is gated by mechanical validation, and no decision is delegated to an opaque large-language model.

The platform is currently running paper trading with a \$103,000 simulated Alpaca account. It is configured for an aggressive 20%+ annualized return target, with the documented trade-off of accepting drawdowns up to 30-35% in adverse regimes. Live trading is structurally gated behind walk-forward validation that the current strategy stack has not yet passed — a feature, not a bug.

2. Investment thesis

What this system is designed to capture

The platform implements three well-documented sources of edge available to disciplined retail capital:

- **Cross-sectional momentum** — names already in established uptrends with elevated participation tend to continue outperforming over 1-3 month horizons. This is the canonical Jegadeesh-Titman effect, widely documented and persistent across decades.
- **Volatility targeting** — sizing positions inversely to recent realized volatility produces materially better risk-adjusted returns (Sharpe / Calmar) without claiming higher raw returns. The honest, repeatable effect is smoother drawdowns for a given exposure.
- **Volatility risk premium** — defined-risk options structures (long calls when IV rank is low, cash-secured puts on names the investor would own anyway) systematically harvest the gap between realized and implied volatility while bounding maximum loss.

What this system is explicitly NOT designed to do

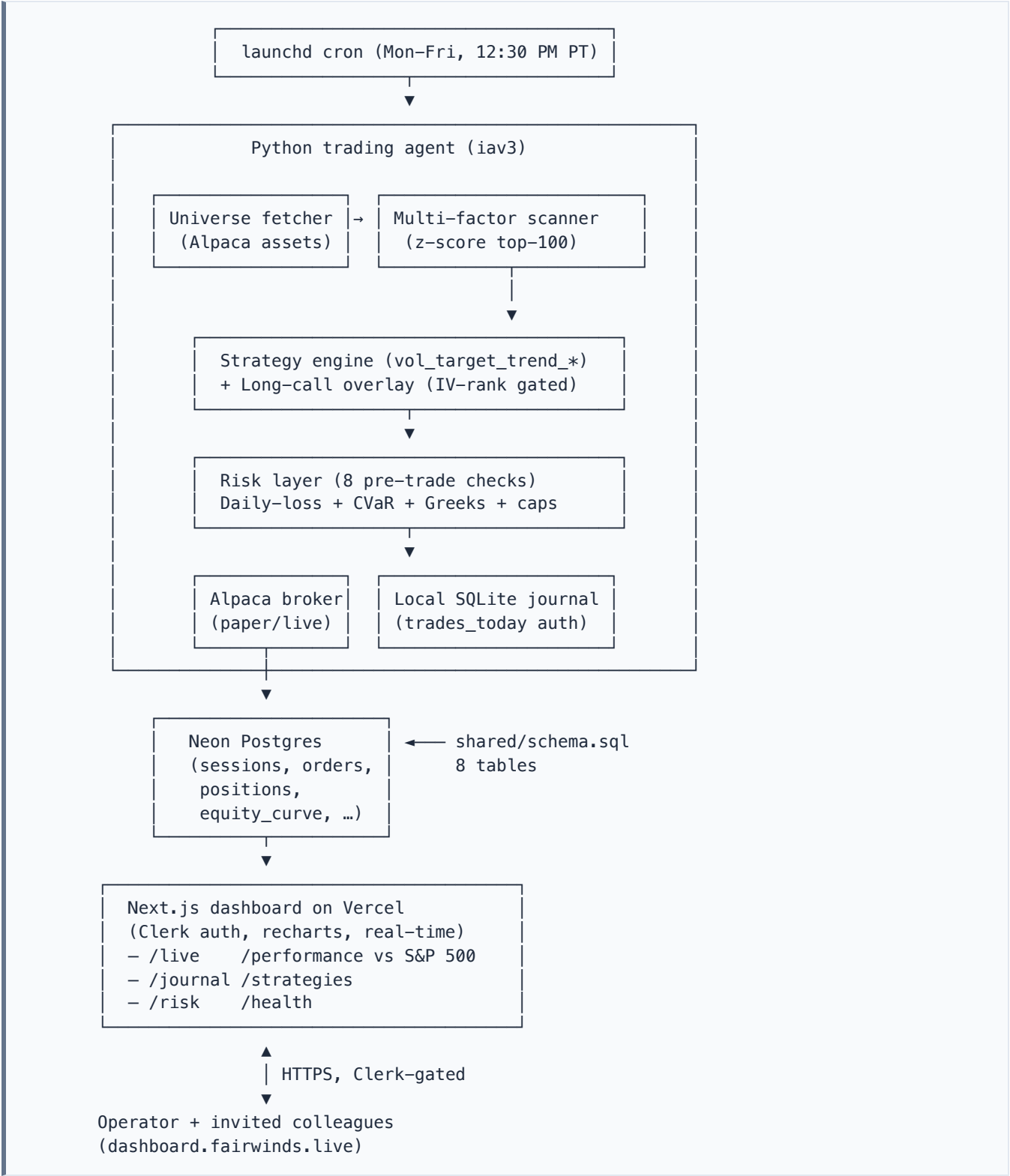
- **Predict markets.** The agent does not forecast prices. It reacts to confirmed signals and bounds its losses when wrong.
- **Trade naked options.** The risk layer structurally rejects naked positions. All option exposure is either long (premium-defined loss) or cash/share-secured (assignment-defined loss).
- **Let an LLM make trading decisions.** Earlier prototypes did this and produced slow, non-deterministic, un-backtestable behavior. The current architecture deliberately keeps Claude out of the decision path. AI is used for post-trade commentary only.
- **Maximize raw return.** The promotion gate explicitly favors risk-adjusted performance (Sharpe > 0.7, max DD < 25%) over absolute return. A strategy that returns 80% in one year and -40% the next does not promote.

Honest framing. Cross-sectional momentum, volatility targeting, and selling vol on names you'd own are well-known patterns. None is novel proprietary alpha. The edge this platform offers comes from *operational discipline*

— atomic bracket orders, walk-forward validation, mechanical risk gates, and complete transparency — applied consistently to known patterns.

3. System architecture

Component diagram



Module responsibilities

MODULE	PATH	RESPONSIBILITY
Universe fetcher	<code>data/universe.py</code>	~4,400 Alpaca-tradeable equities (marginable + shortable + fractionable)
Scanner	<code>data/scanner.py</code>	Multi-factor z-score ranking; picks top-N for this cycle
Strategy	<code>strategy/</code>	vol_target_trend, momentum, mean_reversion + aggressive variant
Options data	<code>data/alpaca_options.py</code>	Real chains, IV rank, Greeks via Alpaca's options API
Wheel	<code>options/wheel_live.py</code>	Cash-secured-put + covered-call decision engine
Overlay	<code>options/long_call_overlay.py</code>	IV-rank-gated long-call overlay on trend signals
Risk	<code>risk/manager.py</code>	Pre-trade checks: cash, concentration, daily loss, CVaR, Greeks
Greeks aggregator	<code>greeks/aggregator.py</code>	Portfolio Δ / Γ / Vega / Θ + hard-cap checker
Backtest	<code>backtest/</code>	Event-driven engine + walk-forward harness
Broker	<code>broker/alpaca.py</code>	Atomic bracket orders + single-leg option orders
Neon journal	<code>db/neon.py</code>	Cloud journal store; dashboard read source
Engine	<code>engine.py</code>	Cycle orchestration: data → signal → risk → execution → journal

4. Trading strategy stack

Layer 1: Dynamic universe selection

Each cycle, the agent fetches the current ~4,400 Alpaca-tradeable US equities (filtered to marginable, shortable, and fractionable — a proxy for genuine liquidity), then ranks them via a four-factor composite z-score:

FACTOR	DEFINITION	CAPTURES
20-day return	$(\text{Close} - \text{Close}_{-21}) / \text{Close}_{-21}$	Short-term momentum
60-day return	$(\text{Close} - \text{Close}_{-61}) / \text{Close}_{-61}$	Medium-term momentum
Trend strength	$(\text{Close} - \text{SMA}_{200}) / \text{SMA}_{200}$	Regime alignment
Volume ratio	Avg vol last 20d / avg vol prior 60d	Participation / confirmation

Each factor is z-scored across the entire universe so they have comparable scale; the four z-scores are summed to a composite, sorted descending, and the top 100 become this cycle's watchlist. Hygiene filters drop penny stocks ($\$ < 5$), reverse-split data corruption ($|\text{60d return}| > 500\%$), and obvious halts (vol ratio $> 50\times$).

Layer 2: Equity entry signal (vol_target_trend_aggressive)

For each scanned name, an entry signal fires when *all three* conditions hold:

- Fast EMA (10-day) is above slow EMA (50-day)
- Close is above the 200-day SMA (regime filter)
- Position not already held

Position size is computed by **inverse-realized-volatility weighting** targeting 20% annualized portfolio volatility, capped at the configured per-name concentration limit (20% of equity). When realized vol on the underlying is low, the strategy sizes up; when realized vol is high, it sizes down — automatically.

Layer 3: Risk-bracketed execution

Every entry is submitted as an atomic **bracket order**: market entry + ATR-based stop-loss + ATR-based take-profit, transmitted as a single Alpaca order so the position is never unprotected. Stop is set $2\times$ ATR below entry; target is $4\times$ ATR above entry (2:1 reward-to-risk).

Layer 4: IV-rank-gated long-call overlay (optional, opt-in)

When `ENABLE_OPTIONS_OVERLAY=true` and the equity trend signal is on for a name, the system additionally consults the long-call overlay. It fires only when:

- IV rank < 50 (calls are historically cheap on this name)
- A liquid 60-DTE ATM call exists (bid > 0 , spread $\leq 20\%$ of mid)
- Sizing math works (12% of equity $\geq$ one contract cost)

When triggered, the system buys defined-risk long calls at limit (mid). Maximum loss is the premium paid; there is no naked exposure. The overlay accumulates IV history per symbol over time, and the gate is dormant for any symbol until 60+ daily IV observations exist — a structural bootstrap protection against acting on insufficient data.

Layer 5: Earnings-event wheel (architecturally ready, deferred)

The `WheelLive` module implements pre-earnings cash-secured-put selling on names the investor would own. It uses Finnhub's earnings calendar to identify positions with earnings in the next 3-14 days, confirms elevated IV, and sells defined-risk CSPs to harvest the volatility risk premium. The decision engine is built and unit-tested; engine integration is deferred pending walk-forward validation of the standalone equity stack.

5. Risk management layers

Every order — equity or option, entry or exit — passes through a pure-function pre-trade check before submission to the broker. The check is deterministic, fully unit-tested, and takes all required state as explicit arguments (no clock reads, no global state, no race conditions).

Pre-trade gates (in evaluation order)

#	GATE	TRIGGER
1	Input sanity	qty > 0, price > 0, portfolio equity > 0
2	SELL-side ownership	Cannot sell what is not held; cannot oversell
3	Absolute daily-loss breaker	Halt new entries if day P&L $\leq$ -6%
4	CVaR daily-loss breaker	Halt if day P&L breaches the mean of the worst 5% of historical days. Arms automatically at 30+ sessions of journal data.
5	Prohibited symbols	Leveraged ETFs (TQQQ, SOXL, UVXY, ...) blocked by policy
6	Per-order cap	Single order $\leq$ \$10,000 notional
7	Daily trade frequency	$\leq$ 20 orders per day (runaway-loop guard)
8	Concentration	Single name $\leq$ 20% of equity post-trade
9	Buying power	Order value $\leq$ available cash
10	Cash reserve	$\geq$ 5% cash retained post-trade
11	Portfolio Greeks (when options active)	$ \Delta < 1500$, $ \text{Vega} < 500$, $\Theta > -100/\text{day}$, notional < 1.5× equity

CVaR breaker — why it matters

An absolute "halt at -3%" cap overshoots in calm regimes and undershoots in high-vol regimes. The Conditional Value-at-Risk (CVaR) breaker compares today's day P&L against the mean of the worst α -fraction of historical days ($\alpha = 5\%$ by default). The threshold auto-adapts: in a low-volatility regime, the breaker becomes tighter; in a high-vol regime, it becomes looser. This prevents both nuisance halts during normal vol expansion and rubber-stamping of genuinely catastrophic days.

The CVaR layer is deliberately dormant until 30+ sessions of journal history exist. Below that threshold, the absolute breaker remains the only daily-loss protection — a documented safety state, not an oversight.

Atomic bracket orders

In the previous version (v2 $\rightarrow$ v3), the agent placed entry orders and then separately called the broker to set stop-loss orders. This created a window — typically 1-3 seconds — during which the position was unprotected. The v3 architecture uses Alpaca's `OrderClass.BRACKET` : entry, stop, and target submitted as one atomic operation. The position cannot exist on the broker's books without its protection levels also in place.

Promotion gate (paper → live)

Live trading is structurally gated. A strategy does not flip from `TRADING_MODE=PAPER` to `TRADING_MODE=LIVE` until:

- Walk-forward aggregate OOS Sharpe > 0.7 (default; loosened to 0.5 for the aggressive configuration)
- Max OOS drawdown $< 25\%$ (default; loosened to 35% for aggressive)
- Worst single OOS window Sharpe > -0.5 (default; loosened to -1.0 for aggressive)
- ≥ 30 sessions of paper trading where realized P&L tracks backtest expectation within 15%
- The operator manually flips the configuration flag

6. A trading day, step by step

Monday through Friday, 12:30 PM Pacific Time (3:30 PM Eastern, 30 minutes before NYSE close):

TIME	STEP	ACTION
12:30:00	1	macOS <code>launchd</code> fires the configured job. Wrapper script activates the Python virtualenv and runs <code>iav3 paper</code> .
12:30:01	2	Health gate: <code>iav3 validate-env</code> pings Alpaca, Finnhub, Anthropic, and Neon. Logs pass/fail for each.
12:30:03	3	Open Neon session row with starting equity, mode, strategy, agent version.
12:30:04	4	Alpaca market clock checked. If closed (weekend / holiday / outside hours), log "market closed" and skip to step 12.
12:30:05	5	Universe fetch — pull ~4,400 tradeable US equities from Alpaca.
12:30:06	6	Scanner — batched bars fetch (100 symbols / call), compute four-factor composite, rank, pick top 100.
12:30:45	7	For each of the 100 top-ranked symbols, in sequence: fetch historical OHLCV, run <code>strategy.generate()</code> , produce trend signal + ATR + target weight.
12:31:30	8	For each new entry candidate: build projected portfolio Greeks, run all 11 pre-trade checks. Blocked orders logged to journal with reason code.
12:31:35	9	Approved orders submitted to Alpaca as atomic bracket orders. Filled at next available price; stop and target levels active immediately.
12:31:40	10	Long-call overlay consulted per symbol with <code>trend=1</code> . If gate passes (<code>trend on AND IV rank < 50 AND liquidity OK AND sizing fits</code>), submit option order at mid limit.
12:31:55	11	Snapshot all open positions to Neon. Record equity point. Compute and log portfolio Greeks.
12:31:58	12	Close Neon session with final equity, cash, day P&L, and full session log. Optionally invoke Claude advisor for plain-English commentary on the cycle.
12:32:00	13	Process exits. Dashboard reflects new state within seconds (Vercel CDN refreshes the page on next view).

Between cycles

Bracket orders placed at Alpaca remain active 24/7 within their day-or-GTC time-in-force. If a position's stop or target is hit between agent cycles (overnight, weekend), Alpaca fills it autonomously. The agent does not need to be running for risk protection to fire.

Failure modes & what happens

FAILURE	SYSTEM BEHAVIOR
Alpaca API down	Cycle aborts; logs the error; no positions affected; next cycle retries

FAILURE	SYSTEM BEHAVIOR
Finnhub rate-limited	Earnings-aware features degrade silently; equity logic continues
Neon DB unreachable	Local SQLite journal still writes; risk-layer reads from local; dashboard shows last cached state
Anthropic API down	Advisor commentary skipped; cycle continues normally
Machine offline	Cycle is missed for that day. Bracket orders at Alpaca remain active. Agent picks up at the next scheduled time.
Risk gate trips	Order is blocked, reason code recorded to journal, cycle continues with remaining symbols

7. Validation methodology

Walk-forward, not in-sample backtest

A traditional in-sample backtest fits parameters to history and reports the resulting performance. This systematically overstates the realistic edge of any strategy because the parameters themselves were chosen with knowledge of the test data. Walk-forward validation breaks this loop:

- Slice history into rolling windows: 36-month in-sample (IS) followed by 6-month out-of-sample (OOS), advanced 6 months at a time.
- For each window: grid-search parameters on the IS data, optimizing in-sample Sharpe.
- Lock in the IS-optimal parameters. Run the strategy on the OOS data with those exact parameters.
- Report aggregate OOS metrics. The strategy has no knowledge of OOS data during parameter selection.

Promotion gate (current configuration)

METRIC	DEFAULT GATE	AGGRESSIVE (CURRENT)	RATIONALE
Aggregate OOS Sharpe	> 0.7	> 0.5	Risk-adjusted return floor
Max OOS drawdown	< 25%	< 35%	Worst-case capital preservation
Worst window Sharpe	> -0.5	> -1.0	Regime robustness (no single-window collapse)

Current baseline (May 2026)

The first formal walk-forward run, conducted on `vol_target_trend` at default parameters across AAPL, MSFT, NVDA, and SPY over 2015 → 2026:

- Aggregate OOS Sharpe: **0.54** (below 0.7 default gate — fails)
- Max OOS drawdown: **24.4%** (below 25% default — passes by a hair)
- Worst window Sharpe: **-3.87** (well below -0.5 floor — fails badly)
- Verdict: **PROMOTION BLOCKED**

The gate worked as designed. A naive backtest of the same strategy reported strong in-sample Sharpe (0.5-2.0 range) that did not survive out-of-sample testing — specifically, the 2019-01 → 2022-01 windows uniformly failed as the COVID + rate-shock regime broke patterns learned 2016-2018. Documented in `docs/walk-forward-baseline-2026-05-22.md` .

Pending validation

The currently active `vol_target_trend_aggressive` configuration (fast EMA 10, slow EMA 50, target vol 0.20) has not yet been walk-forward-tested. The combined system (dynamic scanner + aggressive strategy + long-call overlay) is being validated through paper trading. Before any consideration of a live promotion, the combined system must pass a fresh walk-forward retest with its loosened gates.

8. Technology stack & infrastructure

AGENT (PYTHON)

- Python 3.13
- alpaca-py (broker + market data)
- finnhub-python (earnings, news)
- anthropic (advisor commentary)
- pandas, numpy, scipy (compute)
- psycpg (Neon Postgres)
- pandas-market-calendars (NYSE schedule)
- pytest (183 tests, all passing)

DASHBOARD (TYPESCRIPT)

- Next.js 16 (App Router)
- React 19
- Tailwind CSS v4
- Clerk (authentication)
- @neondatabase/serverless (DB)
- recharts (visualizations)

HOSTING (ALL FREE TIER)

- **Vercel** Hobby — dashboard hosting + CDN
- **Neon** free — 0.5 GB Postgres + 191 compute hrs/mo
- **Clerk** free — up to 10,000 monthly active users
- **GitHub** public repo — unlimited
- **Domain:** `fairwinds.live` at Squarespace
- **Alpaca** paper account — \$0/mo
- **Finnhub** free — 60 calls/min

OPERATIONS

- macOS `launchd` for scheduled cycles
- Logs to `~/Library/Logs/iav3/`
- Real-time dashboard at `dashboard.fairwinds.live`
- Total run cost: **~\$0.02/year** (Anthropic API only)

9. Honest limitations & caveats

- **Survivorship bias.** Historical data sources (yfinance, Alpaca bars) reflect currently-listed companies. Delisted names, bankruptcies, and acquisitions are not in the backtest universe. Real historical returns would be somewhat lower than reported.
- **Slippage model.** Backtests apply a flat 5 basis points of slippage per fill. Real fills, especially on mid-cap names during volatility, will be worse. The actual broker fills via Alpaca's smart order router are typically tighter than this assumption, but cannot be guaranteed.
- **Daily bars only.** The strategy operates on daily-close data, not intraday. Intra-day moves are invisible to the signal generator. This is a deliberate simplicity choice; an intraday redesign is a separate, larger project.
- **Aggressive configuration is not yet walk-forward validated.** The current paper-trading configuration uses tighter parameters and looser gates that have not been retested against history. Validation is pending; current paper data is the bridging evidence.
- **Earnings-event wheel is built but not wired.** The `WheelLive` module is unit-tested and ready, but the engine does not yet invoke it. A separate development cycle is required to integrate, validate, and promote.
- **Single-machine deployment.** The agent runs on the operator's MacBook. If the machine is offline, scheduled cycles are missed. Bracket orders already placed remain active at Alpaca and execute autonomously, but new signals are not acted on until the machine is back online.

- **No tax-loss harvesting, no tax-lot accounting.** The system treats positions as fungible; it does not optimize for short- vs. long-term capital gains. Operators trading in taxable accounts must address this separately.
- **This is not investment advice.** Past performance does not predict future results. Backtests describe historical behavior, not future outcomes. The operator remains solely responsible for all trading decisions and the regulatory consequences of automated trading.

The single most important framing. This platform is engineered to *measure* a strategy on history before risking capital, and to *structurally prevent* the failure modes that destroy retail accounts (unprotected positions, runaway losses, position concentration, naked options, decision-making under cognitive load). It is not a money-printing machine and was never designed to be one. The edge it offers is operational discipline applied consistently — not novel alpha.